Interface for Querying and Data Mining for NYC Yellow and Green Taxi Trip Data

Zahid Aziz
Department of Computer Science
Montclair State University
Montclair, NJ
azizz2@montclair.edu

Stefan Robila
Computational Sensing Laboratory
Department of Computer Science
Montclair State University
Montclair, NJ
robilas@montclair.edu

***Abstract*— Big data analysis is often seen as a complex process for processing large sets of data to uncover hidden information and patterns. This information can then be used by different groups of people to make decisions, to run their businesses and processes more efficiently. Just as important as the analysis technique used, so is the way we present that information. The purpose of our research is to investigate how large data generated by day to day activities in an urban environment can be processed to support analysis and visualization is support of public awareness and decision making. Specifically, New York City's large and regularly updated open data sets focused on taxi rides provide an excellent opportunity for the development of user-friendly web based interface for visualization and querying. Faced with an ever increasing (in both size and diversity) access to open data sets interfaces such as the one we designed nd developed will become valuable tools for public and specialized access (such as journalistic investigation). The application was built using state of the art technologies and provides a visualization of trends in ridership data from the New York City Taxi and Limousine Commission.**

Keywords-Big Data Analysis, Visualization, Taxi and Limusine Commission Data

I. INTRODUCTION

The generation of large data sets continues to expand in both size and diversity at an unprecedented rate [1]. Large amounts of information are collected by commercial companies as well as local and state offices as part of the day to day interaction with their customers. In both private and public sectors, the data are then used as part of bussiness improvement processes, to increase efficiency in services or to identify new potential challeges or initiatives [2]. Yet, use of such massive amounts of data often is faced with skepticism by the general public expressing concerns related to privacy ([3]–[5]) or the lack of transparency with respect to how such data are used [6].

In an effort to counter such negative perceptions, and as a way to increase civic accountability of public institutions to the community they serve, more and more of the data sets are now also made available under an open data model [7]. While the exact definition for open data acess may vary from one national or regional jurisdiction to another, in principle, the model allows for free data access and distribution subject solely to attribution [8].

Access to open data also often comes with interfaces that allows basic interaction. Yet such interfaces may still be perceived with suspicion as they would be associated to the institutions designing and managing them [7]. As an alternate for institution originated interfaces, in this paper, we detail our approach with the design and development of a user friendly tool focused on taxi ride information in New York City. Motivated by a local council enacted legislation, NYC, the largest urban concentration in the United States has made signifficant progress in openning access to its data with over 2,300 streams currently available through a unified web portal [9]. A recent survey on how such data are being used found that open data has generated a variety of maps, tools and apps leadig to creative reuse of data [10].

The choice of data for the project is due to its size and diversity as well as its relevance to current societal debates. NYC's taxi data is large, with millions of data points for each month of data collection and including both time and location information. Coupled with potential access to similar data from ride hailing companies (such as Uber, or Lyft), analysis of such data allows users to understand the impact taxis and car-hire services have on the city, from congestion, revenue, employment, etc, as well as have an insight on availability of transportation services across the city.

II. NYC TAXI AND LIMOUSINE COMMISSION DATA

A. New York City Taxi

At the time of great depression many workers in the NYC area were laid off, and many sought alternative sources of revenue. Many chose to become taxi drivers, and as a result, the number of available cars for hire grew rapidly, outpacing the demand [11]. As a result, the local government enacted regulatory legislation: in 1937, New York City established the taxi cab medallion system.

As part of the medallion system taxi cab would be assigned a piece of metal (attached to the hood of the car) signifying it as a legal cab to operate in the city. To control the supply, NYC is rarely adding any new medallions. This also helped drivers and

cab companies to stay in business, by placing a value on the medallion itself [12]. The NYC Taxi cab business was considered a very profitable investment before hail-ride companies such as Uber and Lyft stepped into the industry. This is evident form the fact that in 2004 a medallion would cost $400,000, which doubled in 2010 and tripled in 2013. As of March 2015, a medallion was sold for $900,000. After the introduction of other rider-hail taxi services like Uber and Lyft, the cost of a medallion has dropped below $160,000 [13].

NYC cab taxi system is regulated by the Taxi and Limousine Commission (TLC) created in 1971. TLC is responsible for licensing and regulating over 50,000 vehicles, including medallions, for-hire vehicles, commuter vans and paratransit vehicles. It also regulates approximately 100,000 drivers [14]. TLC also regulates the fares, determines cab company's lease amount for the driver, monitors the routes to avoid artificial fare inflation by the driver (by capturing the pickup and drop off location coordinates). While the yellow taxis are most known taxis in NYC, various types of taxi services are operated in NYC with their own designated service zones and rules. Yellow and Green taxis can be hailed from the street where as for-hire vehicles (Limos, liveries) can only be ordered by phone or using apps. Whereas yellow taxis can be hailed in any parts of NYC, green (or boro) taxis cannot be hailed in Manhattan (below 96Th street). Green taxis were introduced recently as a mechanism to improve access to ride-hailing served in areas of the city not usually serviced by yellow taxis [15].

B. Prior Work

NYC Taxi trips data has already been analyzed in a variety of projects. Most of the previous researches are related to the Big Data Explanatory Analysis. As part of the explanatory analysis data mining is done based on few data pointers in the trip data. Also, predictive analysis was done to develop a predicting model for different trip variants like tips and ride duration [16]. In another example the taxi data is analyzed based on the hours. This gives us the information about rush hour periods. Work has also been done to build a multi variate regression model for predicting the tip amount for future tips [17]. Different data points were used to analyze the yellow taxi data including number of trips, distance travelled, tip amount, etc.

These examples show explanatory analysis of a large data set. The trip data has been analyzed using several Big Data tools available in the market, including Tableau [18]. The NYC taxi data has also been analyzed in connection with other taxi service providers such as Uber. A comparative analysis of both gives us the understanding how two services are in a competition with each other. This analysis also tries to predict the future growth for Uber and challenges faced by the NYC yellow and Green taxis with increasing competition [19].

In this project, we included explanatory and comparative analysis of NYC taxi data. Explanatory analysis is based on several data pointers including pickup and drop off locations, no of trips, tip and total trip amount. As part of the predictive analysis, we are predicting locations and time of the year for drivers to make more profit. As part of comparative analysis, we are showing how yellow and green taxis share passengers in the boroughs of NYC.

III. PROJECT DATA

A. NYC TLC Taxi Data

The NYC TLC Taxi Data is a large data set that includes information related to every trip taken on yellow and green taxis in NYC. It includes data on pick up and drop off location such as:

- date and time of pick and drop off
- fare amount
- tip amount
- geographical data for pick up and drop off

To protect the privacy of taxi users, the exact geographical locations are not provided in the open data, instead, general blocks of streets are presented. Nevertheless, exact locations can be potentially identified through direct requests (such as the ones under the Freedom of Information Act). However, while such data sets could be located online, they are often not complete and cover only limited time intervals. Compared to them, the TLC released data is provided consistently for each month and in a precise format (CSV).

At the time of writing, the 2018 data is only partially available online [20]. As such, for this research, only 2017 open data was used. For each month, two files in comma separated value (CSV) format are released, one for Yellow Taxi and one for Green Taxi for a total of over 10 million dat points (approximately 1GB in size). With the introduction of other taxi services like Uber, Lyft etc. there is a predicted decrease in the use of Yellow and Green taxis. However, such comparative analysis is out of scope of this project.

B. Data Description

The following provides a detailed description of all fields included in the data set. Complete descriptions for each data file are provide in data dictionaries accessible from the NYC TLC data page [20]:

Taxi Zones - This data set include the available zones where yellow and green taxis operate. The data set include the information about the service zone and the borough a zone belongs to. Below is the sample from Taxi Zone data set (see Fig 1). Note that the zones are provided with names and not with corresponding geographical coordinates.

LocationID	Borough	Zone	service_zone
1	EWR	Newark Airport	EWR
2	Queens	Jamaica Bay	Boro Zone
3	Bronx	Allerton/Pelham Gardens	Boro Zone
4	Manhattan	Alphabet City	Yellow Zone
5	Staten Island	Arden Heights	Boro Zone
6	Staten Island	Arrochar/Fort Wadsworth	Boro Zone

Figure 1. Taxi Zone data set

Taxi Trip Data- This data set is the primary data set used in this project. For this project we used the 2017 data which comprises of 11,740,667 trips for green taxis and 113,496,874 for yellow taxis. This data set includes those attributes that are available as public data. Other data

including the geographical coordinates of pick and drop location are excluded and could only be made available by making a Request of Information (RoI). Fig 2 shows a sample form Taxi Trip Data set.

vendorid integer	lpep_pickup_datetime timestamp without time zone	lpep_dropoff_datetime timestamp without time zone	store_and_fwd_flag character varying(3)	ratecodeid integer	pulocationid integer	dolocationid integer	passenger_count integer	trip_distance character varying(10)
2	2017-01-01 00:01:15	2017-01-01 00:11:05	N	1	42	166	1	1.71
2	2017-01-01 00:03:34	2017-01-01 00:09:00	N	1	75	74	1	1.44
2	2017-01-01 00:04:02	2017-01-01 00:12:55	N	1	82	70	5	3.45
2	2017-01-01 00:01:40	2017-01-01 00:14:23	N	1	255	232	1	2.11
2	2017-01-01 00:00:51	2017-01-01 00:18:55	N	1	166	239	1	2.76

Figure 2. Trip data set

C. Data Extraction and Cleansing

Data extraction was one of the challenges faced at the early stages of this project. As part of extraction we first extracted the data into separate tables for both yellow and green taxies using Postgres's copy command. Postgres is an open source object-relational database management system which uses and extends SQL language combined with other features. It is not controlled by one organization as it's free and open source system [21].

```
Copy GreenTaxiData_import2 from 'C:\MS Project\2017_green_Taxi_Trip_Data.csv' DELIMITER ',' CSV HEADER;
```

Figure 3. Command to import data

The command shown in Fig. 3 extracts the entire data from csv file and puts it into the table in Postgres. The table should already exist with the same fields in the same order as are in the .csv file. After the successful extraction, the data was cleansed by removing the incorrect and incomplete data and was imported into a single table. Data cleansing is done based on the trip distance and total trip amount. As part of data cleansing records with missing or zero trip distance or trip amount, were not imported. A dataset with monthly grouping is then generated based on pickup and drop off locations. This data set is the main source of the data for this project.

This dataset finds the average trip amount, average tip amount and number of trips form and to the location per month for each taxi type (Green & Yellow). Grouping the data in this format would help us quickly retrieve the despised data compared to if it was stored individually.

IV. Application Architecture Overview

The application consists of three major components as described in the architecture diagram in Fig 4. Following, we discuss the steps taken in the design and implementation of the application components.

A. Application User Interface (UI)

The application UI is developed using React, Bootstrap, Foundation and Redux store.

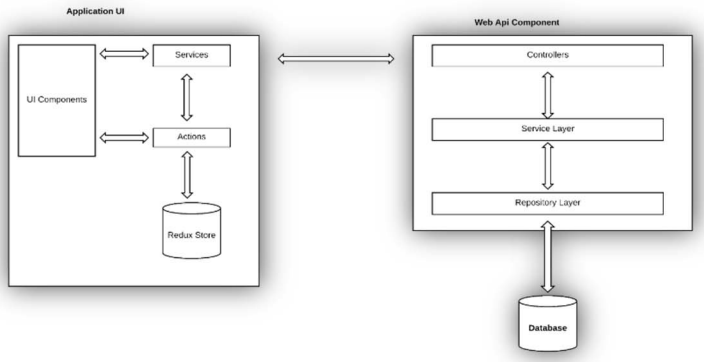

Figure 4. Application Architecture Overview

React is a front-end library for developing user interfaces developed by Facebook. It allows us to create reusable, composable and stateful components [22]. It allows us to create UI components just like function in JavaScript which can then be called anywhere. We can call their properties and state. We can reuse these smaller components to build bigger components. React is declarative in nature, which means we only need to tell what to do and then it takes care of how to do it.

Bootstrap is an open source tool kit for building UI component using HTML, CSS and JS. It allows us to create flexible and responsive UI components [23]. Advantages of using bootstrap include saving time for development (due to automatic code generation), high degree of customization, and high responsiveness and consistency across majority of the browsers.

Foundation is front-end frame work with several built-in templates, controls and utilities to make more responsive UI components [24]. Using foundation allows the user to reuse the already available template and controls. We can always customize those template and controls are per our requirements.

Redux is a state container for JavaScript applications [25]. It helps us write applications that behave consistently under different environments and react to changes in their state. As Single Page Applications (SPAs) are getting popular, it is becoming harder to manage the state of a complex application. The application state store information including server responses, cached data, locally created data which needs to be managed on the UI. Redux help us manage this ever-changing application state by storing the state in a single source of truth.

B. UI Components

The application UI is divided into several components. Each component is organized into following sub components

- *View*- This represents the presentation layer of a component. It consists of HTML, CSS and JSX scripts. Views are reactive to any change in the application state. View is implemented as a react component. It contains a render function which is rendered every time the view is loaded, or the application state is changed.
- *Container* - Container acts like a bridge between the View and the store. Container connects the view with store by

mapping view's properties to application's state and the store.

- *Service* - This part of the component provides service to the view by doing heavy lifting of pulling the data from API calls. Separating the service from views gives us extra flexibility to process the information before it is handed over to the view. Also, it provides an abstraction level as well.
- *Action* - Information is sent consistently between the application and the store using actions. An action is a payload of information which sends the data to store. In Redux actions are triggered using the "dispatch" function of the store. Action is the only source of information for store.
- *Reducer*- Reducers specify how the application state should be changed after an action has triggered. Each component has a reducer which is invoked in response to an action updating the current state of the application. An action specifies only "what" has changed where as a reducer specifies "how" it has changed. Reducer directly access and updates application's state which in turn renders the view if any view is attached to that specific instance of application state.

Each of the UI component consists of a separate file for each of the sub component mentioned above. Each sub component is responsible for a single task to achieve separation of concerns as explained above. For example, a view contains code that deals with the component's presentation. These sub components are interdependent to function together. Fig 5 displays the diagram showing communication between these sub components.

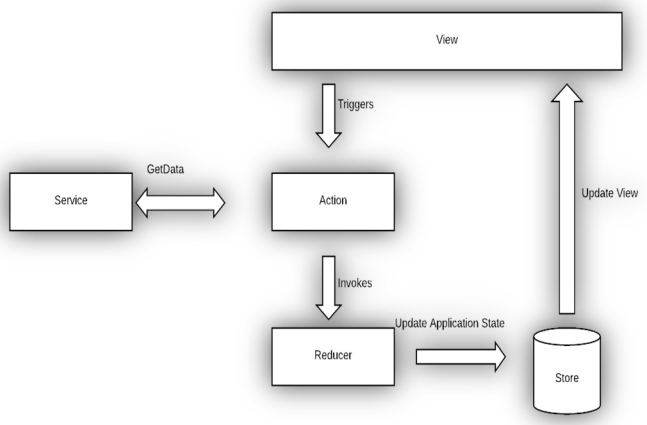

Figure 5. UI Subcomponent communication model

C. Redux Store

Redux store is a state management container. The store is initialized in the "store.js" file. Below the initialization statement. Initial state of the store is an empty object. The store maintains the application state as an object tree.

```
const createStore = (initialState = {})
```

Middlewares and reducers are also initialized in the store

D. Web API Component

The WebAPI component exposes the APIs which exposes the data to the outside world. These APIs are developed in .Net WebAPI [26] (see Fig 6).

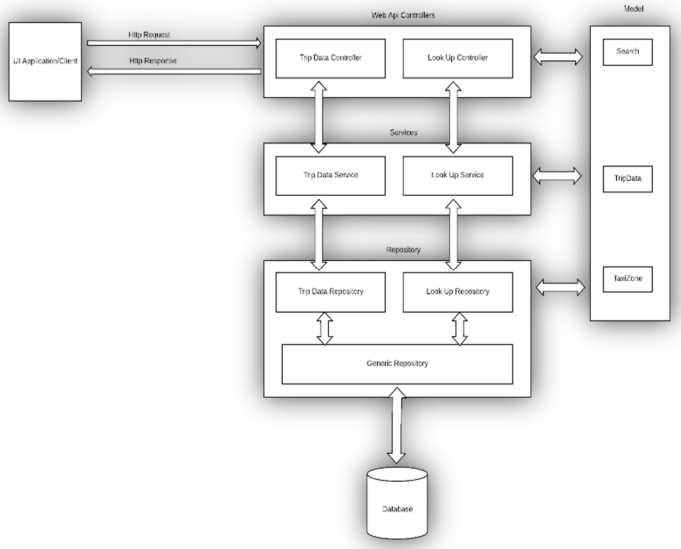

Figure 6. Web API Component Architecture

This layered architecture allows for ease of development. The *WebApi controllers* are the endpoint exposed through different routes defined in the configuration files. The TripData controllers handles the data related to the trips while the Lookup Controller's role is to expose the look up data. *The Services layer* represents the service layer of the application. This layer provides data to the controllers and constitutes an abstraction between them and the repository. This abstraction layer accomplishes the separation of concern between different components of the system. *The Repository layer* is responsible for direct interaction with the data source. It is internally divided into service specific repositories and generic repositories. Service specific repositories serve the corresponding service in the service layer whereas generic repository interacts with the data source. Generic repository retrieves the required data and returns it to the service specific repository which in turn returns it back to the service. Finally, the *Models* are used to transform and transport the data between different layers. Models are plain non-framework specific classes representing, not necessary all, entities in the system.

In addition, Dependency Injection (DI) in Object Oriented Programing (OOP) design is the process of providing an object as a dependency to another object in other to use its services. DI is achieved by creating a builder component that injects the dependency into another component whenever it is required. The injected object is also the component of the same application. For this project, simple injectors were installed using the NuGet Manager [27].

Finally, routing, i.e. the process of directing the incoming client request to appropriate controller was defined in the WebApi configuration file. We can define multiple routes in this

file. A default route is defined with optional parameters as below. Each route defined here should uniquely identify one route.

E. Database Setup

The last component but an important one for this project is the database. It holds the actual data that is being exposed by the WebAPI and consumed by the UI Application. For this we used open source Postgres as a backend database [21]. Npgsql is used as a data provider to connect to Postgres. Npgsql is an open source data provider written in C# to access Postgres [28]. It can be installed using the NuGet Package manager. A connection string was defined in the web.config file to create a connection between WebAPI and the Database.

A complete installation guide is available online together with the source code and data for both the client side [29] and the WEB [30] APIs. These references also provide access to the Github repositories.

V. RESULTS

Based on the functionality the application could be divided into three major components: Dashboard, Location and Chart View.

The *Dashboard* is the default view of the application as shown in Fig 7. The dashboard displays two data grids one for each yellow and green taxi type trip data.

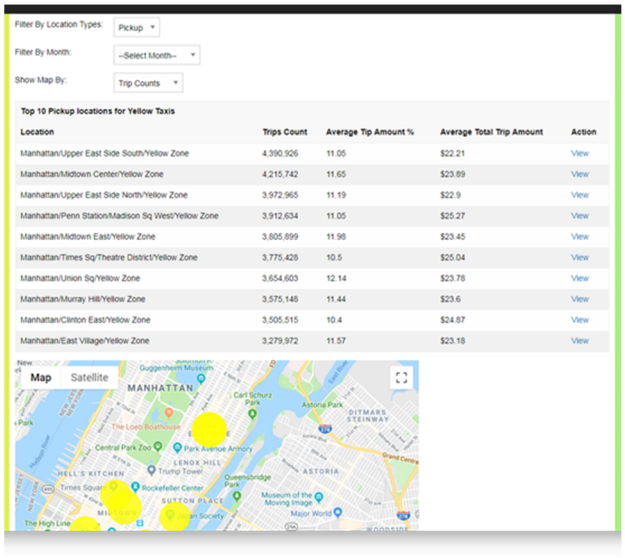

Figure 7. Dashboard view

By default, the displayed trip data is ordered by trip counts. It shows top 10 pickup locations with most trip counts. Grid data could be filtered by location type (pickup or drop off), trip Month(s) and by average tip amount or average total trip amount. The data is always shown in descending order. The trip data could further be filtered by month and/or by trip counts, average tip amount or average trip amount.

The Dashboard view also contains a map showing the locations contained in the data grid. Since we do not have the exact coordinates of trip locations, Google map API is used to get the coordinates based on the location's Borough and service zone name. These coordinates are then saved to the Database for future references.

Once the desired data is retrieved on the dash board, user can click on the view link in the grid to get details for that location. The *Location* detail view displays the top 3 locations based on the filter selection in the dashboard view. If the selected location type in the dashboard is a pickup location, the location detail view would display the top 3 destinations for the selected location (see Fig. 8). The destinations are ranked by the number of trips from the current location. These top 3 locations are also displayed on the Google map using the geographical coordinates for those locations.

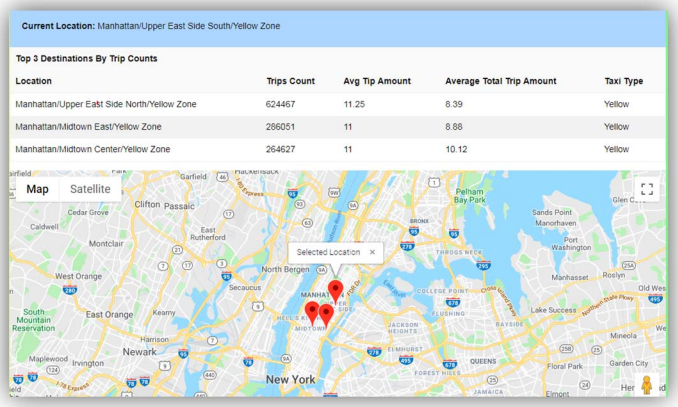

Figure 8. Location Detail

The *Chart* view contains the charts to show comparison between trip data for Yellow and Green taxies. Chart view contains bar chart and line chart. By default, both the charts show data by monthly trip counts as shown in the Fig 9 and 10.

Figure 9. Bar chart showing monthly trip counts

These charts can be filtered by pickup location(s), month(s) and/or trip count, tip amount or trip amount. Below is the bar chart filtered for pick up location "Queens/Boro Zone/Jamaica", for selected months and by tip amount (see Fig 11).

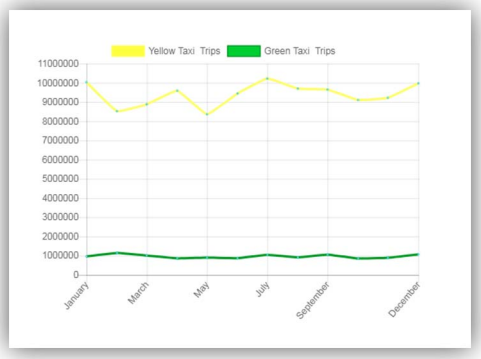

Figure 10. Line chart showing monthly trip counts

Figure 11. Bar chart filtered by location, month and tip amount

VI. FUTURE WORK

This project's aim was to develop a web-based user interface to analyze and visualize a large dataset of NYC Taxi data. The dataset used in this project includes only the Yellow and Green taxi trip data. This web-based user interface (UI) allows the user to filter and visualize data on factors like trip counts for a location, average tip amount and average total trip amount. While designed as a proof of concept on how multiple technologies can be combined to foster access to open data, the system has broader impacts, As examples, it could be used by the riders to find out areas with finding taxis in short time, by the drivers to find out what areas are good in terms of finding rides with high tip amounts. or by city planners to find a popular destination at specific time of the year.

The system integrates seamlessly with the NYC TLC released data and could be easily expandable to account for other data streams such as liveries and potentially ride-hailing company data if such data would become available and follow the TLC format. Future developments can also include direct API connectivity with the NYC TLC data website (thus eliminating the need of human in the loop for future data downloads) as well as integration of additional streams (such as weather) that will enable further data analysis and visualization.

REFERENCES

[1] J. Zander and P. Mähönen, "Riding the data tsunami in the cloud: myths and challenges in future wireless access," IEEE Commun. Mag., vol. 51, no. 3, pp. 145–151, 2013.

[2] I. A. T. Hashem, I. Yaqoob, N. B. Anuar, S. Mokhtar, A. Gani, and S. U. Khan, "The rise of 'big data' on cloud computing: Review and open research issues," Inf. Syst., vol. 47, pp. 98–115, 2015.

[3] L. Van Zoonen, "Privacy concerns in smart cities," Gov. Inf. Q., vol. 33, no. 3, pp. 472–480, 2016.

[4] L. Xu, C. Jiang, J. Wang, J. Yuan, and Y. Ren, "Information security in big data: privacy and data mining," Ieee Access, vol. 2, pp. 1149–1176, 2014.

[5] O. Tene and J. Polonetsky, "Privacy in the age of big data: a time for big decisions," Stan Rev Online, vol. 64, p. 63, 2011.

[6] M. Andrejevic, "Big data, big questions| the big data divide," Int. J. Commun., vol. 8, p. 17, 2014.

[7] M. Janssen, Y. Charalabidis, and A. Zuiderwijk, "Benefits, adoption barriers and myths of open data and open government," Inf. Syst. Manag., vol. 29, no. 4, pp. 258–268, 2012.

[8] S. S. Dawes, L. Vidiasova, and O. Parkhimovich, "Planning and designing open government data programs: An ecosystem approach," Gov. Inf. Q., vol. 33, no. 1, pp. 15–27, 2016.

[9] "NYC Open Data -." [Online]. Available: https://opendata.cityofnewyork.us/. [Accessed: 17-Mar-2019].

[10] K. Okamoto, "What is being done with open government data? An exploratory analysis of public uses of New York City open data," Webology, vol. 13, no. 1, p. 1, 2016.

[11] G. R. G. Hodges, Taxi!: a social history of the New York City cabdriver. JHU Press, 2007.

[12] W. Skok, "Knowledge management: New York City taxi cab case study," Knowl. Process Manag., vol. 10, no. 2, pp. 127–135, 2003.

[13] W. Hu, "Taxi Medallions, Once a Safe Investment, Now Drag Owners Into Debt," The New York Times, 22-Dec-2017.

[14] "Taxi & Limousine Commission." .

[15] T. Mann, "Mayor's Taxi Plan Gets Green Light," Wall Street Journal, 07-Jun-2013.

[16] Kaggle, "NYC Taxi Trips - Exploratory Data Analysis | Kaggle." [Online]. Available: https://www.kaggle.com/kartikkannapur/nyc-taxi-trips-exploratory-data-analysis. [Accessed: 17-Mar-2019].

[17] I. Khatri, "The Data Science of NYC Taxi Trips: An Analysis & Visualization." .

[18] L. Nair, S. Shetty, and S. Shetty, "Interactive visual analytics on Big Data: Tableau vs D3. js," J. E-Learn. Knowl. Soc., vol. 12, no. 4, 2016.

[19] T. Schneider, Analyzing 1.1 Billion NYC Taxi and Uber Trips, with a Vengeance. 2015.

[20] TLC, "TLC Trip Record Data." [Online]. Available: https://www1.nyc.gov/site/tlc/about/tlc-trip-record-data.page. [Accessed: 17-Mar-2019].

[21] M. Stonebraker and G. Kemnitz, "The POSTGRES next-generation database management system," Commun. ACM, vol. 34, no. 10, pp. 78–93, 1991.

[22] C. Staff, "React: Facebook's functional turn on writing Javascript," Commun. ACM, vol. 59, no. 12, pp. 56–62, 2016.

[23] J. Spurlock, Bootstrap: Responsive Web Development. O'Reilly Media, Inc., 2013.

[24] A. Shenoy, Introducing Zurb Foundation 6. Springer, 2016.

[25] M. D. Abramoff, P. J. Magalhaes, and S. J. Ram, "Image processing with ImageJ," Biophotonics Int., vol. 11, no. 7, pp. 36–43, 2004.

[26] J. Kurtz and B. Wortman, ASP. NET Web API 2: Building a REST Service from Start to Finish. Apress, 2014.

[27] D. Arh and D. Dakić, NuGet 2 Essentials. Packt Publishing Ltd, 2013.

[28] H. Schwichtenberg, "Installing Entity Framework Core," in Modern Data Access with Entity Framework Core, Springer, 2018, pp. 15–29.

[29] Z. Aziz, "NYCTAXIClientSide," 05-Dec-2018. [Online]. Available: https://github.com/zeemsu/NYCTAXIClientSide. [Accessed: 17-Mar-2019].

[30] Z. Aziz, “NYCTAXI-WebApi,” 02-Dec-2018. [Online]. Available: https://github.com/zeemsu/NYCTAXI-WebApi. [Accessed: 17-Mar-2019].